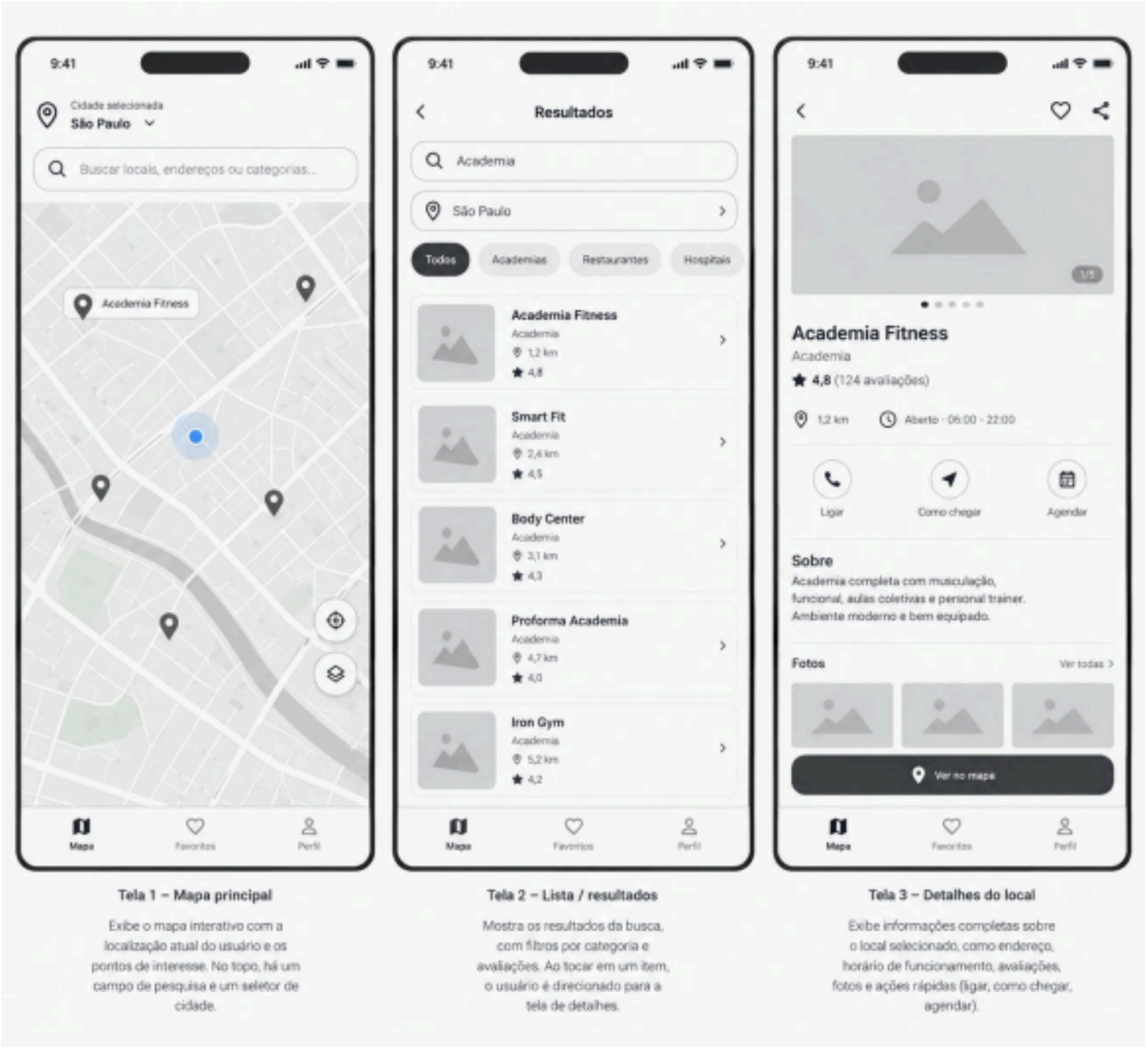
AVALIAÇÃO FORMATIVA + TUTORIAL

Desenvolvimento de Aplicação Mobile com Flutter

Aplicação de localização: mapa, pesquisa, resultados e detalhes

Sua aplicação deverá consistir no seguinte WireFrame:



SENAI

1. Proposta da avaliação

Você deverá desenvolver uma aplicação mobile utilizando Flutter e Firebase Cloud Firestore. A aplicação será baseada no wireframe apresentado e terá como finalidade permitir que o usuário pesquise estabelecimentos, selecione uma cidade, visualize resultados e consulte os detalhes de um local.

Diferentemente de uma aplicação que utiliza apenas dados fixos no código, nesta avaliação os locais deverão

ser armazenados no Cloud Firestore. Dessa forma, a aplicação deverá buscar, cadastrar, alterar e excluir registros.

O projeto será desenvolvido em três telas principais. Recursos adicionais, como cadastro e edição, poderão aparecer em diálogos ou Bottom Sheets, sem que seja necessário criar uma quarta tela principal.

2. Tecnologias obrigatórias

Tecnologia	Uso	Obrigatório
Flutter / Dart	Interface e lógica da aplicação	Sim
Firebase	Backend da aplicação	Sim
Cloud Firestore	Banco de dados NoSQL	Sim
FlutterFire CLI	Configuração do Firebase no Flutter	Sim
Google Maps ou outro mapa real	Mapa real	Não — extensão

3. O banco de dados utilizado

O banco de dados desta atividade será o Cloud Firestore, banco de dados NoSQL da plataforma Firebase. Os dados são organizados em coleções e documentos. Para esta aplicação, a principal coleção será chamada locais.

```
locais
  ├── documento_1
  │   ├── nome: "Academia Fitness"
  │   ├── categoria: "Academia"
  │   ├── cidade: "São Paulo"
  │   ├── avaliacao: 4.8
  │   ├── distancia: 1.2
  │   ├── horario: "06:00 - 22:00"
  │   ├── descricao: "..."
  │   └── favorito: false
  │
  ├── documento_2
  │   └── ...
  │
  └── documento_3
  └── ...
```

O identificador de cada documento poderá ser gerado automaticamente pelo Firestore. A documentação oficial do Firebase apresenta os métodos set(), add() e update() para gravação e atualização de documentos. Na implementação Flutter, o acesso é feito por meio do pacote cloud_firestore.

4. Situação-problema

Uma empresa deseja disponibilizar um aplicativo que permita aos usuários localizar serviços e estabelecimentos em diferentes cidades. O usuário precisa selecionar a cidade, pesquisar o que procura, filtrar os resultados, abrir os detalhes de um estabelecimento e marcar locais como favoritos.

O sistema deverá permitir que novos locais sejam cadastrados e que registros existentes sejam alterados ou excluídos.

5. Telas obrigatórias

Tela	Função	Elementos	Ações
1 — Mapa principal	Entrada da aplicação	Cidade, pesquisa, mapa/representação, marcadores	Selecionar cidade, pesquisar, abrir resultados
2 — Resultados	Apresentar locais do Firestore	Pesquisa, filtros, lista, botão de cadastro	Filtrar, abrir, editar, excluir, cadastrar
3 — Detalhes	Apresentar um local	Imagem, nome, categoria, avaliação, horário, descrição, favorito	Favoritar, voltar, ver no mapa

6. Requisitos funcionais

Código	Requisito	Descrição
RF01	Selecionar cidade	Selecionar ao menos três cidades.
RF02	Pesquisar	Pesquisar por nome ou categoria.
RF03	Consultar Firestore	Os resultados devem vir do Cloud Firestore.
RF04	Filtrar	Filtrar por categoria e cidade.
RF05	Cadastrar local	Criar documento na coleção locais.
RF06	Editar local	Alterar dados de um documento existente.
RF07	Excluir local	Excluir um documento após

		confirmação.
RF08	Detalhes	Exibir informações do local selecionado.
RF09	Favoritar	Alterar o campo favorito do documento.
RF10	Navegação	Navegar entre mapa, resultados e detalhes.

7. Requisitos não funcionais

- A aplicação deverá ser desenvolvida em Flutter/Dart.
- O banco deverá ser Cloud Firestore.
- O projeto deverá estar configurado com FlutterFire.
- A interface deverá funcionar em Android.
- O código deverá ser organizado em pastas.
- A aplicação não deverá apresentar erros durante a demonstração.
- Operações de exclusão deverão pedir confirmação.
- A interface deverá apresentar mensagens de sucesso ou erro nas operações de banco.

8. Critérios de avaliação — 100 pontos

Item	Critério	Valor
01	Configuração Flutter + Firebase funcionando	10
02	Tela principal e seleção de cidade	7.5
03	Consulta de dados do Firestore	10
04	Pesquisa e filtros	10
05	CRUD — cadastro	10
06	CRUD — edição	7.5
07	CRUD — exclusão	7.5
08	Tela de detalhes	7.5
09	Favoritos	5

10	Navegação e usabilidade	5
11	Organização/qualidade do código	10
12	Testes e documentação	10
	TOTAL	100

9. Tutorial — preparação do ambiente

O tutorial abaixo utiliza o fluxo recomendado atualmente pela documentação do Firebase para Flutter: Firebase CLI + FlutterFire CLI + firebase_core + cloud_firestore.

9.1 Criar o projeto Flutter

```
flutter create mapa_locais
cd mapa_locais
code .
```

Teste o projeto antes de instalar qualquer integração:

```
flutter run
```

9.2 Criar o projeto no Firebase

1. Acesse o Firebase Console e entre com uma conta Google.
2. Crie um novo projeto.
3. Escolha um nome para o projeto, por exemplo: mapa-locais-flutter.
4. Conclua a criação do projeto.
5. Não é necessário criar o banco manualmente fora do Firebase Console; ele será habilitado na etapa seguinte.

9.3 Instalar o Firebase CLI

Instale a Firebase CLI seguindo o procedimento oficial para seu sistema operacional. Depois faça login:

```
firebase login
```

Verifique se o comando funciona:

```
firebase --version
```

9.4 Instalar o FlutterFire CLI

```
dart pub global activate flutterfire_cli
```

Verifique:

```
flutterfire --help
```

9.5 Configurar o projeto Flutter com Firebase

Dentro da pasta do projeto:

```
flutterfire configure
```

Selecione o projeto Firebase criado e a plataforma Android. O FlutterFire criará o arquivo lib/firebase_options.dart. Esse arquivo contém configurações específicas do projeto; não altere manualmente seus valores.

9.6 Instalar os pacotes

```
flutter pub add firebase_core
flutter pub add cloud_firestore
```

Depois, se houver alteração de configuração:

```
flutterfire configure
```

9.7 Inicializar o Firebase

Substitua o conteúdo principal de lib/main.dart por uma estrutura semelhante a:

```
import 'package:flutter/material.dart';
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
import 'screens/mapa_screen.dart';

Future<void> main() async {
 WidgetsFlutterBinding.ensureInitialized();

 await Firebase.initializeApp(
 options: DefaultFirebaseOptions.currentPlatform,
 );

 runApp(const MinhaAplicacao());
}

class MinhaAplicacao extends StatelessWidget {
 const MinhaAplicacao({super.key});

 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 debugShowCheckedModeBanner: false,
 title: 'Mapa de Locais',
 theme: ThemeData(
 colorScheme: ColorScheme.fromSeed(
 seedColor: Colors.blue,
 ),
 useMaterial3: true,
 ),
 home: const MapaScreen(),
 );
 }
```

```
}
```

10. Criar o Cloud Firestore

No Firebase Console, abra o produto Cloud Firestore e crie o banco de dados. Para fins didáticos, a configuração inicial pode ser feita no modo de teste durante o desenvolvimento, mas o projeto final deverá possuir regras de segurança adequadas. Nunca trate regras abertas de desenvolvimento como configuração de produção.

10.1 Criar a coleção locais

Crie a coleção:

```
locais
```

Cadastre manualmente alguns documentos para testar a leitura. Sugestão de campos:

```
nome String
categoria String
cidade String
avaliacao Number
distancia Number
horario String
descricao String
favorito Boolean
```

11. Estrutura recomendada do projeto

```
lib/
├── main.dart
├── firebase_options.dart
├── models/
│   └── local.dart
├── services/
│   └── local_service.dart
├── screens/
│   ├── mapa_screen.dart
│   ├── resultados_screen.dart
│   └── detalhes_screen.dart
├── widgets/
│   ├── local_card.dart
│   └── local_form.dart
└── dialogs/
    └── confirm_delete.dart
```

12. Criar o Model Local

```
class Local {
 final String id;
 final String nome;
```

```
 final String categoria;
 final String cidade;
 final double avaliacao;
 final double distancia;
 final String horario;
 final String descricao;
 final bool favorito;

 Local({
 required this.id,
 required this.nome,
 required this.categoria,
 required this.cidade,
 required this.avaliacao,
 required this.distancia,
 required this.horario,
 required this.descricao,
 required this.favorito,
 });

 factory Local.fromMap(String id, Map<String, dynamic> data) {
return Local(
 id: id,
 nome: data['nome'] ?? '',
 categoria: data['categoria'] ?? '',
 cidade: data['cidade'] ?? '',
 avaliacao: (data['avaliacao'] ?? 0).toDouble(),
 distancia: (data['distancia'] ?? 0).toDouble(),
 horario: data['horario'] ?? '',
 descricao: data['descricao'] ?? '',
 favorito: data['favorito'] ?? false,
 );
 }

 Map<String, dynamic> toMap() {
 return {
 'nome': nome,
 'categoria': categoria,
 'cidade': cidade,
 'avaliacao': avaliacao,
 'distancia': distancia,
 'horario': horario,
 'descricao': descricao,
 'favorito': favorito,
 };
 }
}
```

13. Criar o serviço de banco de dados

A classe LocalService será responsável por conversar com o Firestore. Isso evita colocar comandos de banco diretamente nas telas.

```
import 'package:cloud_firestore/cloud_firestore.dart';
import '../models/local.dart';

class LocalService {
 final CollectionReference locais =
 FirebaseFirestore.instance.collection('locais');

 Stream<List<Local>> listarLocais() {
 return locais.snapshots().map((snapshot) {
 return snapshot.docs.map((doc) {
 return Local.fromMap(
 doc.id,
 doc.data() as Map<String, dynamic>,
 );
 }).toList();
 });
 }

 Future<void> adicionar(Local local) async {
 await locais.add(local.toMap());
 }

 Future<void> atualizar(Local local) async {
 await locais.doc(local.id).update(local.toMap());
 }

 Future<void> excluir(String id) async {
 await locais.doc(id).delete();
 }

 Future<void> alterarFavorito(
 String id,
 bool favorito,
 ) async {
 await locais.doc(id).update({
 'favorito': favorito,
 });
 }
}
```

14. Entendendo o CRUD

Operação	Firestore	Método	Objetivo
Create	locais.add()	POST lógico	Cadastrar local
Read	locais.snapshots()	GET lógico	Listar locais
Update	locais.doc(id).update()	PUT/PATCH lógico	Editar local/favorito

Delete	locais.doc(id).delete()	DELETE lógico	Excluir local

15. Ler os dados no Flutter

Um exemplo simples de lista conectada ao Firestore:

```
final LocalService service = LocalService();

StreamBuilder<List<Local>>(
 stream: service.listarLocais(),
 builder: (context, snapshot) {
 if (snapshot.connectionState == ConnectionState.waiting) {
 return const Center(
 child: CircularProgressIndicator(),
 );
 }

 if (snapshot.hasError) {
 return const Center(
 child: Text('Erro ao carregar os locais.'),
 );
 }

 final locais = snapshot.data ?? [];

 if (locais.isEmpty) {
 return const Center(
 child: Text('Nenhum local encontrado.'),
 );
 }

 return ListView.builder(
 itemCount: locais.length,
 itemBuilder: (context, index) {
 final local = locais[index];

 return ListTile(
 title: Text(local.nome),
 subtitle: Text(local.categoria),
 onTap: () {
```

SENAI

```
 // abrir detalhes
 },
 );
 },
 );
 },
)
```

16. Implementar pesquisa e filtros

Para a primeira versão didática, o Firestore pode fornecer os documentos e o Flutter pode aplicar os filtros em memória. Isso simplifica a compreensão. Em uma versão mais avançada, o aluno poderá estudar consultas do Firestore com where(), orderBy() e índices.

```
List<Local> filtrarLocais(
 List<Local> locais,
 String texto,
 String cidade,
 String categoria,
) {
 return locais.where((local) {
 final correspondeCidade =
 cidade == 'Todas' || local.cidade == cidade;

 final correspondeCategoria =
 categoria == 'Todos' ||
 local.categoria == categoria;

 final termo = texto.toLowerCase();

 final correspondeTexto =
 local.nome.toLowerCase().contains(termo) ||
 local.categoria.toLowerCase().contains(termo);

 return correspondeCidade &&
 correspondeCategoria &&
 correspondeTexto;
 }).toList();
}
```

17. Cadastrar um local

Como a atividade mantém três telas principais, o cadastro pode ser feito por meio de um showModalBottomSheet ou AlertDialog. Crie campos para nome, categoria, cidade, horário, avaliação, distância e descrição.

SENAI

```
Future<void> cadastrarLocal() async {
 final novoLocal = Local(
 id: '',
 nome: nomeController.text,
 categoria: categoriaSelecionada,
 cidade: cidadeSelecionada,
 avaliacao: double.tryParse(
 avaliacaoController.text,
 ) ?? 0,
 distancia: double.tryParse(
```

```
 distanciaController.text,
 ) ?? 0,
 horario: horarioController.text,
 descricao: descricaoController.text,
 favorito: false,
 );

 await LocalService().adicionar(novoLocal);
}
```

Após salvar, a lista será atualizada automaticamente se a tela estiver utilizando snapshots() como no exemplo anterior.

18. Editar um local

Ao selecionar editar, abra o mesmo formulário preenchido com os dados atuais e chame:

```
await LocalService().atualizar(localEditado);
```

O documento será identificado pelo seu ID do Firestore. Essa é uma das razões para manter o campo id no Model.

19. Excluir um local

Nunca exclua diretamente ao tocar no botão. Primeiro peça confirmação:

```
final confirmou = await showDialog<bool>(
 context: context,
 builder: (context) {
 return AlertDialog(
 title: const Text('Excluir local?'),
 content: const Text(
 'Essa ação não poderá ser desfeita.',
 ),
 actions: [
 TextButton(
 onPressed: () => Navigator.pop(context, false),
 child: const Text('Cancelar'),
```




```
 ),
 FilledButton(
 onPressed: () => Navigator.pop(context, true),
 child: const Text('Excluir'),
 ),
 ],
 );
 },
);

if (confirmou == true) {
 await LocalService().excluir(local.id);
}
```

20. Implementar favoritos

Na tela de detalhes, o ícone de coração deverá alterar o campo favorito do documento.

```
await LocalService().alterarFavorito(
 local.id,
 !local.favorito,
);
```

Para a atividade básica, os favoritos podem ser exibidos como um filtro dentro da tela de resultados.

Como extensão, o aluno pode criar uma quarta tela de Favoritos.

21. Navegação entre as três telas

```
Navigator.push(
 context,
 MaterialPageRoute(
 builder: (_) => ResultadosScreen(
 cidade: cidadeSelecionada,
 pesquisa: pesquisaController.text,
 ),
 ),
);
```

Para abrir detalhes:

```
Navigator.push(
 context,
 MaterialPageRoute(
 builder: (_) => DetalhesScreen(
 local: local,
 ),
 ),
);
```

22. Tela principal — requisitos de implementação

• Exibir cidade selecionada.

• Permitir selecionar São Paulo, Campinas e Jundiaí, no mínimo.

• Possuir campo de pesquisa.

• Possuir representação de mapa.

• Exibir pelo menos cinco marcadores visuais.

• Possuir botão para acessar resultados.

• Manter aparência próxima ao wireframe.

23. Tela de resultados — requisitos de implementação

• Ler os locais diretamente do Firestore.

• Exibir nome, categoria, distância e avaliação.

• Possuir pesquisa.

• Possuir filtro por categoria.

• Possuir filtro por cidade.

• Permitir abrir detalhes.

• Permitir cadastrar local.

• Permitir editar local.

• Permitir excluir local.

24. Tela de detalhes — requisitos de implementação

• Receber o local selecionado.

• Exibir todas as informações cadastradas.

• Exibir botão de favorito.

• Permitir alterar o favorito no Firestore.

• Possuir botão 'Ver no mapa'.

• Possuir botão/ação de retorno.

25. Regras de segurança do Firestore

Durante o desenvolvimento, o modo de teste pode facilitar a aprendizagem. Porém, a entrega final deverá demonstrar que o aluno entende que o banco precisa ser protegido. Como extensão, implemente autenticação e regras que permitam operações administrativas apenas para usuários autenticados.

O aluno não deverá entregar um projeto final deixando deliberadamente o banco aberto para qualquer pessoa.



26. Testes obrigatórios

ID	Teste	Procedimento	Resultado
T01	Conexão	Abrir o aplicativo e verificar se os dados do Firestore aparecem.	Passou/Falhou
T02	Pesquisa	Pesquisar por 'Academia'.	Passou/Falhou
T03	Cidade	Selecionar Campinas.	Passou/Falhou
T04	Filtro	Selecionar uma categoria.	Passou/Falhou

T05	Create	Cadastrar um novo local e verificar no Firestore.	Passou/Falhou
T06	Read	Fechar/reabrir a tela e verificar o registro.	Passou/Falhou
T07	Update	Editar nome ou horário e confirmar a alteração.	Passou/Falhou
T08	Delete	Excluir um local e confirmar que desapareceu.	Passou/Falhou
T09	Favorito	Marcar e desmarcar um local.	Passou/Falhou
T10	Navegação	Abrir resultados e detalhes e retornar.	Passou/Falhou
T11	Erro	Simular ausência de dados/erro e verificar mensagem.	Passou/Falhou



27. Documentação que deverá ser entregue

• Descrição da problemática.

• Objetivo da aplicação.

• Requisitos funcionais.

• Requisitos não funcionais.

• Tecnologias utilizadas.

• Descrição da arquitetura do projeto.

• Modelo da coleção locais.

• Explicação das operações CRUD.

• Tabela de testes.

• Capturas das três telas.

• Print do Firestore mostrando os documentos cadastrados.

• Conclusão sobre o desenvolvimento.

28. Desafios extras

Nível	Desafio	Descrição
Nível 1	Adicionar imagens	Usar Firebase Storage para armazenar imagens dos estabelecimentos.
Nível 2	Mapa real	Substituir a representação por Google Maps ou outra solução de mapas.
Nível 3	Autenticação	Criar login e separar usuários comuns de administradores.
Nível 4	Consultas Firestore	Realizar filtros diretamente no Firestore com where/orderBy.
Nível 5	Favoritos por usuário	Cada usuário possui sua própria lista de favoritos.
Nível 6	Avaliações	Permitir que usuários registrem avaliações.



29. Checklist de entrega

- ☐ Projeto Flutter executando.
- ☐ Firebase conectado ao projeto.
- ☐ Cloud Firestore configurado.
- ☐ Coleção locais criada.
- ☐ Pelo menos 10 locais cadastrados.
- ☐ Tela de mapa implementada.
- ☐ Tela de resultados implementada.
- ☐ Tela de detalhes implementada.

- ☐ Leitura do Firestore funcionando.
- ☐ Cadastro funcionando.
- ☐ Edição funcionando.
- ☐ Exclusão funcionando.
- ☐ Favoritos funcionando.
- ☐ Pesquisa funcionando.
- ☐ Filtros funcionando.
- ☐ Navegação funcionando.
- ☐ Testes documentados.
- ☐ Prints das telas anexados.
- ☐ Print do Firestore anexado.

31. Orientação final

Desenvolva o projeto por etapas. Primeiro conecte Flutter ao Firebase e confirme que consegue ler um documento. Depois implemente a tela de resultados. Em seguida faça o cadastro, edição e exclusão. Só depois finalize detalhes, favoritos, filtros e acabamento visual. Ao encontrar um erro, identifique se ele está na interface, no Model, no serviço ou na configuração do Firebase.



32. Referências técnicas oficiais

A configuração utilizada neste tutorial segue o fluxo documentado oficialmente pelo Firebase para Flutter: Firebase CLI, FlutterFire CLI, firebase_core e plugins específicos como cloud_firestore. Consulte a documentação oficial antes de adaptar comandos para versões futuras.

Firebase para Flutter: https://firebase.google.com/docs/flutter/setup

Cloud Firestore — operações de dados: https://firebase.google.com/docs/firestore/manage-data/add-data